计算机系统 I：指令集体系结构 (Instruction Set Architecture) 复习讲义

目录

1 ISA 基础 (Foundations of ISA) 2
1.1 ISA 在计算机系统中的位置 (System Stack) 2
1.2 什么是指令 (What is an Instruction) 2
1.3 操作数的位置 (Where Are the Operands) 3
1.4 指令格式与指令集 (Instruction Formats & Instruction Set) 3
1.5 ISA 的定义与机器状态 (ISA Specifies State Change) 3
1.6 ISA 像“方言” (Different ISAs as Dialects) 4
1.7 指令格式设计思路 (Thoughts of Instruction Formats) 4
1.8 四条设计原则 (Design Principles) 4

2 ISA 设计问题 (Issues in ISA Design) 5
2.1 基本设计问题 (Basic Design Issues) 5
2.2 设计权衡决策 (Design Tradeoff Decisions) 5

3 操作数个数 (Number of Operands) 5
3.1 为什么很少见四地址指令 5
3.2 三操作数指令 (Three-Operand Instructions) 6
3.3 二操作数指令 (Two-Operand Instructions) 6
3.4 一操作数指令 (One-Operand Instructions) 7
3.5 零操作数指令与栈 (Zero-Operand Instructions and Stack) 7
3.6 操作数个数的取舍 8

4 寻址方式 (Addressing Modes) 8
4.1 有效地址 (Effective Address) 8
4.2 立即寻址 (Immediate Addressing) 8
4.3 直接寻址 (Direct Addressing) 8
4.4 间接寻址 (Indirect Addressing) 8

4.5 寄存器直接与寄存器间接 (Register Direct / Indirect) 9
4.6 相对、变址与基址寻址 (Relative, Indexed and Based) 9
4.7 寻址方式总结表 (Summary of Addressing Modes) 9
4.8 寻址示例 (Addressing Example) 9

5 操作类型与指令编码 (Operations and Encodings) 10
5.1 操作类型 (Types of Operations) 10
5.2 典型操作集合 (Typical Operations) 10
5.3 编码类型 (Types of Encodings) 11

6 ISA 演化与 CISC/RISC (Evolution and CISC/RISC) 11
6.1 指令集的演化 (Evolution of Instruction Sets) 11
6.2 CISC：复杂指令集计算机 12
6.3 RISC：精简指令集计算机 13
6.4 典型 RISC 特征 (A Typical RISC) 13
6.5 MIPS 指令格式示例 13
6.6 CISC 与 RISC 对比 14
6.7 CISC/RISC 的现实复杂性 14
6.8 现代 x86 与近期趋势 15

7 ISA 分类与操作数位置 (Classification of ISAs) 15
7.1 按操作数位置分类 15
7.2 代码序列：C = A + B 在四类指令集中的表示 16
7.3 为什么几乎所有新架构都使用 GPR 16
7.4 栈架构 (Stack Architectures) 17
7.5 累加器架构 (Accumulator Architectures) 17
7.6 内存-内存架构 (Memory-Memory Architectures) 17
7.7 寄存器-内存架构 (Register-Memory Architectures) 18
7.8 装载-存储架构 (Load-Store Architectures) 18
7.9 寄存器的优缺点 (Registers: Advantages and Disadvantages) 19

8 复习抓手与补充知识 (Review Notes) 19
8.1 ISA 与微体系结构的区别 19
8.2 评估 ISA 的常用指标 19
8.3 常见易混点 20

9 全章总结 (Summary) 20

1 ISA 基础 (Foundations of ISA)

1.1 ISA 在计算机系统中的位置 (System Stack)

指令集体系结构 (Instruction Set Architecture, ISA) 是软件与硬件之间最重要的接口。课件给出的层次可以概括为：

层次	**说明**
Application	应用程序，面向最终用户的功能。
Compiler / Libraries	编译器、宏库、过程库，把高级语言映射到机器相关代码。
Operating System	操作系统，管理资源并提供运行环境。
Instruction Set Architecture	**软硬件契约：软件可见的机器接口**。
Computer Organization / Microarchitecture	ISA 的具体实现方式，如流水线、缓存、执行单元组织。
RTL / Digital Logic / Device Technology	寄存器传输级、数字逻辑电路和器件技术。

ISA 的核心定位

- ISA 位于操作系统和微体系结构之间，是软件能直接“看见”的硬件抽象。
- ISA 定义机器语言；汇编语言是机器语言的人类可读表示。
- 同一个 ISA 可以有多个不同实现。例如同属 x86 ISA 的 Xeon 和 Celeron 可以采用不同微体系结构。

1.2 什么是指令 (What is an Instruction)

一条指令由**操作码 (opcode)** 和 **操作数 (operands)** 组成。指令告诉处理器要做什么、对什么数据做、结果放在哪里，以及下一步执行哪里。

- **Operation Code (Opcode)**：指定操作类型，例如加、减、装载、跳转。
- **Source Operand Reference(s)**：源操作数位置，即“对谁操作”。
- **Result Operand Reference(s)**：结果操作数位置，即“答案放哪里”。
- **Next Instruction Reference**：下一条指令的位置。通常由程序计数器 (PC) 自动递增得到，分支和跳转指令会改变它。

1.3 操作数的位置 (Where Are the Operands)

操作数可以来自多个地方：

- **Main Memory**：主存中的某个地址。
- **CPU Register**：CPU 内部寄存器，速度快、编号短。
- **I/O Device**：输入输出设备。
- **Instruction Itself**：指令中直接携带的立即数。

为了说明“哪个寄存器、哪个内存位置、哪个 I/O 设备”，ISA 必须设计对应的**寻址方式 (addressing scheme)**。

1.4 指令格式与指令集 (Instruction Formats & Instruction Set)

指令格式是机器码中各个字段的安排方式，设计时通常需要考虑：

- **Instruction Length**：短指令、长指令或可变长指令。
- **Number of Operands**：0、1、2 或 3 个显式操作数。
- **Number of Addressable Registers**：可编码多少寄存器。
- **Memory Organization**：内存按字节寻址还是按字寻址。
- **Addressing Modes**：直接、间接、变址等寻址方式如何编码。

指令集 (Instruction Set) 是 CPU 能理解的全部指令集合。它最终以二进制机器码，也称目标代码 (object code)，存在于程序中；程序员通常用汇编语言表示这些机器码模式。

1.5 ISA 的定义与机器状态 (ISA Specifies State Change)

ISA 不只是“指令列表”，还包括程序员可见的机器视图：

- 指令集本身。
- 寄存器数量、名称和角色。
- 内存组织方式，例如地址长度、字节序、字节/字寻址。
- 异常和中断处理模型。

从状态机角度看，一条指令把计算机从一个机器状态转换到另一个机器状态：

$$\text{After State} = \text{Instruction}(\text{Before State})$$

机器状态至少包含：

- **PC**：程序计数器，指向当前或下一条指令。
- **Memory State**：主存内容，例如按字节寻址、小端序、32 位地址空间。
- **Register State**：寄存器文件内容，例如 $R0$ 到 $R31$，每个 32 位。

因此，指令格式、指令类型、寻址方式共同决定一次状态变化如何发生。

1.6 ISA 像“方言” (Different ISAs as Dialects)

不同 ISA 可以完成同样的计算任务，但表达方式不同，类似不同地区的方言。早期 von Neumann 体系结构思想指出：从形式逻辑上说，某些简单指令集已经足以控制任意运算序列；但实际选择 ISA 时，决定性因素是硬件简单性、程序表达清晰性和执行速度。也就是说，**理论上能做**不等于**工程上好用**。

1.7 指令格式设计思路 (Thoughts of Instruction Formats)

课件列出的指令格式设计建议可以整理为：

- 指令长度尽量短，减少取指带宽与存储空间。
- 为未来新操作码预留足够位数，避免 ISA 过早耗尽编码空间。
- 编码要清晰、可区分，便于硬件译码和文档说明。
- 合理选择操作数个数。
- 注意指令对齐，例如字节级或字级对齐。
- 尽量保持规则性：固定指令长度、固定操作码位数、固定操作数字段位置等。

1.8 四条设计原则 (Design Principles)

Hennessy 和 Patterson 总结了 ISA 设计中反复出现的四条原则：

ISA 设计四原则

1. **Simplicity favors regularity**：简单性偏好规则性。例如固定指令大小、固定格式、统一数据格式能简化硬件。
2. **Make the common case fast**：让常见情况更快。常用数据应尽量放在寄存器中，减少读、移动和写的位数。
3. **Smaller is faster**：更小通常更快。小常数很常见，因此立即数字段可以较小；寄存器编号字段也远短于内存地址字段。
4. **Good design demands good compromises**：好设计需要折中。重要例外可能需要特殊格式，例如跳转到很远目标时，小立即数字段不够用。

2 ISA 设计问题 (Issues in ISA Design)

2.1 基本设计问题 (Basic Design Issues)

ISA 设计必须回答以下问题：

- **Data Types**：支持哪些数据类型？大小是多少？
- **Operations**：提供多少操作？每个操作有多复杂？理论上 `LD/ST/INC/BRN`，甚至“减法 + 分支”也能编码任意计算，但程序会很长，实用性差。
- **Operands**：每条指令如何说明操作数？大多数操作是二元操作，例如 $A \leftarrow B + C$；也有一元操作，例如 $A \leftarrow\sim B$。
- **Operand and Result Locations**：操作数和结果能否在内存中？有多少显式操作数？内存操作数如何定位？哪些操作数必须在寄存器中？
- **Instruction Encoding**：如何把操作码、寄存器编号、立即数和地址编码成一致的指令格式？

典型指令集常假设：32 位字长、32 位基本地址、32 位整数等基本操作数；一般情形下可能要引用 3 个操作数，即 $A := B + C$。核心挑战是在较少位数中编码足够多的信息。

2.2 设计权衡决策 (Design Tradeoff Decisions)

不同 ISA 会在以下维度上做取舍：

- **Operation Repertoire**：操作数量、能力和复杂度。
- **Data Types**：操作作用于整数、浮点数、字符串、向量还是特殊领域数据。
- **Registers**：寄存器数量、寄存器角色，以及哪些操作能直接作用于寄存器。
- **Addressing**：地址如何指定，是立即数、直接地址、寄存器、间接地址还是变址地址。

3 操作数个数 (Number of Operands)

3.1 为什么很少见四地址指令

以加法为例，如果要把两个内存位置相加，需要两个源地址；结果还需要一个目的地址；下一条指令若也显式指定，则还要第四个地址。若这些都是内存地址，指令会非常长。

实践中几乎不会看到四地址指令，原因是：

- CPU 控制逻辑和译码复杂度过高。
- 指令字太长，取指成本高。

- 所有操作数并非总是频繁使用。
- 下一条指令通常由 PC 递增得到，只有分支或跳转才显式改变 PC。

因此，大多数 ISA 使用 1、2 或 3 个操作数地址。下面统一用

$$Y = \frac{A - B}{C + (D \times E)}$$

比较不同操作数个数的表达方式。

3.2 三操作数指令 (Three-Operand Instructions)

三操作数指令可以同时指定两个源操作数和一个目的操作数：

指令	含义
`SUB R1, A, B`	$R1 \leftarrow A - B$
`MUL R2, D, E`	$R2 \leftarrow D \times E$
`ADD R2, R2, C`	$R2 \leftarrow R2 + C$
`DIV R1, R1, R2`	$R1 \leftarrow R1 / R2$

三地址格式的优点是灵活，结果位置可显式指定；原始的 A, B, C, D, E 不会被破坏。代价是每条指令需要编码更多操作数字段，指令可能更长。

3.3 二操作数指令 (Two-Operand Instructions)

二操作数指令通常把第一个操作数同时作为源和目的，即结果覆盖第一个操作数：

指令	含义
`SUB A, B`	$A \leftarrow A - B$
`MUL D, E`	$D \leftarrow D \times E$
`ADD D, C`	$D \leftarrow D + C$
`DIV A, D`	$A \leftarrow A / D$

这样指令较短，但会破坏 A 和 D 的原始值。如果后续还需要原值，必须先复制：

指令	目的
`MOV R1, A`	保存 A 的副本。
`MOV R2, D`	保存 D 的副本。
`SUB R1, B`	$R1 \leftarrow R1 - B$
`MUL R2, E`	$R2 \leftarrow R2 \times E$
`ADD R2, C`	$R2 \leftarrow R2 + C$
`DIV R1, R2`	$R1 \leftarrow R1 / R2$

3.4 一操作数指令 (One-Operand Instructions)

一操作数指令把另一个操作数隐含在累加器 (Accumulator, ACC) 中：

指令	含义
`LDA D`	$ACC \leftarrow D$
`MUL E`	$ACC \leftarrow ACC \times E$
`ADD C`	$ACC \leftarrow ACC + C$
`STO R1`	$R1 \leftarrow ACC$
`LDA A`	$ACC \leftarrow A$
`SUB B`	$ACC \leftarrow A - B$
`DIV R1`	$ACC \leftarrow ACC / R1$

早期计算机大量使用一地址指令，因为 CPU 设计简单；但程序表达会变得更繁琐，累加器也容易成为瓶颈。

3.5 零操作数指令与栈 (Zero-Operand Instructions and Stack)

零操作数指令通常基于栈。栈是后进先出 (LIFO) 的数据结构：

- `PUSH A`：把 A 放入栈顶。
- `POP A`：弹出栈顶元素并放入 A。
- `ADD`：弹出栈顶两个元素，相加后把结果压回栈顶。

对同一个表达式，可以使用如下栈序列：

指令	栈内容变化
`PUSH B`	B
`PUSH A`	B, A
`SUB`	$A - B$
`PUSH E`	$(A - B), E$
`PUSH D`	$(A - B), E, D$
`MUL`	$(A - B), E \times D$
`PUSH C`	$(A - B), E \times D, C$
`ADD`	$(A - B), E \times D + C$
`DIV`	$(A - B)/(E \times D + C)$

3.6 操作数个数的取舍

- **更多操作数**：指令更强，程序指令数更少，但指令长、硬件复杂。
- **更多寄存器**：寄存器间操作更快，也便于编译器保存临时变量。
- **更少操作数**：指令短、取指和执行可能更快，但程序指令数更多，表达复杂。

4 寻址方式 (Addressing Modes)

4.1 有效地址 (Effective Address)

寻址方式说明一条指令的操作数字段如何引用真正想要的数据。它可以指定常量、寄存器或内存位置。操作数真实所在位置称为**有效地址 (effective address, EA)**。某些寻址方式允许在运行时动态计算 EA，这对数组遍历、指针和过程调用非常重要。

4.2 立即寻址 (Immediate Addressing)

立即寻址中，操作数字段直接包含要使用的值：

```
ADD #5
```

表示把常数 5 加到某个隐含或显式目的操作数上。数字 5 的补码表示直接存放在指令中。立即数必须在汇编时已知，优点是无需访问内存取操作数。

4.3 直接寻址 (Direct Addressing)

直接寻址中，操作数字段给出内存地址：

```
ADD 100
```

表示取内存地址 100 中的内容参与运算，即 $M[100]$。缺点是完整地址必须放入指令，可能限制地址空间。例如 32 位指令字若还要容纳 32 位地址，就几乎没有空间放操作码和其他字段。常见解决办法包括只编码偏移量，或使用隐含段寄存器。

4.4 间接寻址 (Indirect Addressing)

间接寻址中，操作数字段给出的地址里保存了真正数据的地址：

`ADD [100]` $\Rightarrow$ $M[M[100]]$

缺点是需要额外一次内存访问；优点是第二级地址可在运行时改变，适合动态访问不同地址，例如遍历数组或处理指针。间接寻址也可以看成取指周期之外增加了一个寻址子周期。

4.5 寄存器直接与寄存器间接 (Register Direct / Indirect)

- **Register Direct**：`ADD R5` 表示取寄存器 $R5$ 的内容。寄存器数量有限，编号字段短，不需要像内存地址那样占用很多位。
- **Register Indirect**：`ADD [R3]` 表示 $R3$ 中保存的是内存地址，先到该地址取数据，再参与运算，即 $M[R3]$。

4.6 相对、变址与基址寻址 (Relative, Indexed and Based)

- **Relative Addressing**：以 PC 为基准，加上指令中的偏移量得到 EA。常用于分支跳转，因为目标通常离当前位置不远。
- **Indexed Addressing**：使用一个索引寄存器作为偏移量，与指令中的地址相加得到 EA，适合数组访问。
- **Based Addressing**：使用基址寄存器保存基地址，指令中的地址字段表示相对该基地址的位移。
- **Indexed 与 Based 的区别**：索引寄存器更像“相对给定地址的偏移”，基址寄存器更像“区域起点”，指令字段是相对起点的 displacement。

4.7 寻址方式总结表 (Summary of Addressing Modes)

Addressing Mode	**Syntax**	**Meaning**
Immediate	$\#K$	K
Direct	K	$M[K]$
Indirect	(K)	$M[M[K]]$
Register	(Rn)	$M[Rn]$
Register Indexed	$(Rm + Rn)$	$M[Rm + Rn]$
Register Based	$(Rm + X)$	$M[Rm + X]$
Register Based Indexed	$(Rm + Rn + X)$	$M[Rm + Rn + X]$

课件强调了四类常见地址计算来源：汇编时已知常数、寄存器内容、两个寄存器之和、寄存器和常数之和。实际 ISA 会给这些组合命名为不同寻址方式。

4.8 寻址示例 (Addressing Example)

设内存和寄存器状态如下：

$$M[800] = 900, \quad M[900] = 1000, \quad M[1000] = 500,$$
$$M[1100] = 600, \quad M[1600] = 700, \qquad R1 = 800.$$

对装入累加器的 `Load` 指令，各寻址方式得到：

寻址方式	**形式**	**装入 AC 的值**
Immediate	`Load 800`	800
Direct	`Load 800`	$M[800] = 900$
Indirect	`Load 800`	$M[M[800]] = M[900] = 1000$
Indexed	`Load R1[800]`	$M[R1 + 800] = M[1600] = 700$

如果写成 `Load R1`，在寄存器直接寻址下会装入 $R1$ 的值 800；若是寄存器间接寻址 `Load [R1]`，则会装入 $M[800] = 900$。

5 操作类型与指令编码 (Operations and Encodings)

5.1 操作类型 (Types of Operations)

ISA 通常包含以下操作类型：

- **Arithmetic and Logic**：整数、十进制或浮点加减乘除，逻辑与、或、非等。
- **Shift**：左移、右移、循环左移、循环右移。
- **Data Transfer**：`MOV`、`LOAD`、`STORE`。
- **String**：字符串搜索、转换等。
- **Control**：`BRANCH`、`JMP`、`CALL`、`RET`。
- **System**：`HALT`、打开/关闭中断、模式切换等。
- **Input/Output**：输入、输出。

5.2 典型操作集合 (Typical Operations)

自 1960 年代以来，主流 ISA 的操作类型变化不大：

类别	典型操作
Data Movement	从内存装载、存入内存、内存到内存移动、寄存器到寄存器移动、I/O 输入输出、栈的 push/pop。
Arithmetic	整数或浮点的 Add、Subtract、Multiply、Divide。
Shift	左/右移，左/右循环移位。
Logical	not、and、or、set、clear。
Control	无条件跳转和条件分支。
Subroutine Linkage	call、return。
Interrupt	trap、return。
Synchronization	test-and-set 等原子读-改-写操作。
String / Graphics	字符串搜索与转换；MMX 等并行子字操作，例如 4 个 16 位数据并行相加。

5.3 编码类型 (Types of Encodings)

指令编码大体可分为：

- **Variable Length**：可变长指令。优点是代码密度高，适合强调代码大小的场景；缺点是译码复杂。
- **Fixed Length**：固定长指令。优点是取指、译码和流水线更简单，性能更容易做高；缺点是短操作也占完整指令宽度。
- **Hybrid**：混合长度。兼顾性能和代码密度。

若**代码大小**最重要，倾向使用可变长指令；若**性能**最重要，倾向使用固定长指令。近代嵌入式机器如 ARM、MIPS 增加了可选的 16 位子集模式，例如 Thumb 和 MIPS16，可按过程选择性能或密度。有些架构还探索运行时解压缩指令，以提升代码密度。

6 ISA 演化与 CISC/RISC (Evolution and CISC/RISC)

6.1 指令集的演化 (Evolution of Instruction Sets)

课件给出的演化路径可概括为：

阶段	代表与特点
Single Accumulator	EDSAC, 1950, Maurice Wilkes。以单累加器为核心。
Accumulator + Index Registers	Manchester Mark I、IBM 700 系列，1953。加入索引寄存器改善地址计算。
Separation of Programming Model from Implementation	程序员模型与内部实现逐渐分离。
High-level Language Based	B5000, 1963。面向高级语言支持。
Concept of a Family	IBM 360, 1964。一个 ISA 家族支持多种实现。
General Purpose Register Machines	通用寄存器机器成为主流基础。
Complex Instruction Sets	VAX、Intel 432，1977–1980，走向 CISC。
Load/Store Architecture	CDC 6600、Cray 1，1963–1976，走向 RISC。
CISC / RISC	CISC 代表 x86、Pentium；RISC 代表 MIPS、SPARC、HP-PA、IBM RS6000、PowerPC 等。

6.2 CISC：复杂指令集计算机

CISC (Complex Instruction Set Computers) 的主要思想是缩小编程语义与机器执行之间的差距：

- 指令更复杂，能直接表达较高级操作。
- 代码尺寸更小，这在内存昂贵的时代很重要。
- 编译器可能更简单，因为复杂工作可交给指令和微码。
- 机器内部常有另一个由微码控制的状态机。
- 典型例子：x86、Intel 432、IBM 360、DEC VAX。

x86 例子

- `MOVSD` 是移动双字的 1 字节指令，语义可理解为 $m32[DS:EDI] \leftarrow m32[DS:ESI]$。
- `REP` 是 1 字节前缀，用于重复字符串操作，例如 `REP MOVSD`，重复次数由 `ECX` 设置。
- `LOCK ADD ds:[esi+ecx*2+0x67452301], 0xEFCDAB89` 可编码为 13 字节，包含前缀、复杂地址计算和 32 位立即数，体现了 CISC 的复杂编码能力。

6.3 RISC：精简指令集计算机

RISC (Reduced Instruction Set Computer) 的出发点来自 IBM 的观察：可用指令很多，但程序实际频繁使用的指令很少。

- CISC 可能出现“$n+1$”现象：加入一条新指令若需要额外译码逻辑，可能拖慢整个 ISA 的译码路径。
- RISC 起源于 IBM 1975 年的电话项目，目标是达到 12 MIPS，用于每秒约 300 次呼叫、每次约 20k 条指令。
- IBM 801 于 1978 年出现，是早期 RISC 代表。
- RISC 依靠简单指令和更强编译器来换取性能。

6.4 典型 RISC 特征 (A Typical RISC)

- 指令数量较少。
- 固定格式指令，例如 32 位。
- 三地址、寄存器到寄存器算术指令。
- 执行阶段常追求单周期操作。
- Load-store 架构：只有 load/store 访问内存，ALU 操作只在寄存器之间进行。
- 简单寻址方式，例如 base + displacement。
- 无间接寻址或极少使用复杂间接寻址。
- 分支条件简单。
- 硬布线控制，无微码。
- 需要更多编译器优化工作。
- 代表：Berkeley RISC I/II、Stanford MIPS、IBM RISC Technology、Sun SPARC、HP PA-RISC、ARM。

6.5 MIPS 指令格式示例

MIPS 是典型 RISC ISA，课件列出 5 类格式：

格式	字段	例子
R-format	Op, Rs, Rt, Rd, Shamt, Funct	`add $1, $2, $3`
I-format (Register-Immediate)	Op, Rs, Rt, immediate	`addi $1, $2, -5`
I-format (Load/Store)	Op, Base, Dest, immediate	`lw $1, 24($9)`
I-format (Branch)	Op, Rs, Rt, immediate	`beq L1, $4, $0`
J-format	Op, target	`j L2`

6.6 CISC 与 RISC 对比

CISC	RISC
可变长指令。	固定长指令，常追求单周期操作。
指令和寻址方式丰富。	指令和寻址方式较少。
译码长且复杂。	译码简单。
可包含内存到内存操作。	Load/store 架构。
使用微码。	无微指令，由硬件逻辑直接译码执行。
语义差距小，把复杂度转移到微码。	需要聪明编译器，或硬件动态重排指令。
IBM 360、DEC VAX、x86、Motorola 68030。	IBM 801、MIPS、RISC I、IBM POWER、Sun SPARC。

6.7 CISC/RISC 的现实复杂性

早期机器的现实数据并非完全二分：

CISC/过渡代表	IBM 370/168	VAX 11/780	Xerox Dorado
Year	1973	1978	1978
# Instructions	208	303	270
Microcode	54KB	61KB	17KB
Instruction Size	2–6B	2–57B	1–3B
Execution Model	Reg-reg/Reg-mem/Mem-mem	Reg-reg/Reg-mem/Mem-mem	Stack

RISC 代表	IBM 801	Berkeley RISC1	Stanford MIPS
Year	1980	1981	1983
# Instructions	120	39	55
Microcode	0	0	0
Instruction Size	4B	4B	4B
Execution Model	Reg-reg	Reg-reg	Reg-reg

Colwell 等人与 Patterson、Hennessy 等人围绕 CISC/RISC 的争论说明：**CISC/RISC 不应被看成严格二分法**。例如 DEC 的 MicroVAX-32 是单芯片实现，子集化 VAX 指令但仍有 175

条指令，能模拟复杂指令，具有可变长指令、非 load/store 机器、微码控制和 VAX 寻址方式。它到底算 RISC 还是 CISC，并不容易一句话判定。

6.8 现代 x86 与近期趋势

现代 x86 常被称为“CISC 外衣，RISC 内核”：

- Intel P6、Netburst、Core，AMD Athlon、Phenom、Opteron 等都会把 x86 指令动态译码成 μop 或 RISC-op。
- 内部微体系结构采用很多 RISC 设计思想。
- 处理器动态调度 μop，提高并行度。
- 编译器调度仍然有价值，因为硬件重排并不能替代所有静态优化。

近期 ISA 趋势体现为“按应用增加复杂性”：

- MIPS 中也可能出现复杂指令，例如 `CABS.LE.PS $fcc0, $f8, $f10`，用于比较复数/向量绝对值关系。
- 无线通信和 DSP 中会加入 Viterbi 指令。
- SSE 或 DSP 中常见 sum of absolute differences (PSAD)，用于 MPEG 运动估计：$C = \sum |A - B|$。
- 嵌入式领域代码大小很关键。
- 为降低编程工作量、优化性能，可能使用专用硬件加速器、协处理器，或可扩展 ISA plug-in。

7 ISA 分类与操作数位置 (Classification of ISAs)

7.1 按操作数位置分类

课件按照 ALU 操作数主要来自哪里，把 ISA 分成几类：

类别	典型指令	含义与代表
Accumulator	`add A`	1 地址，$acc \leftarrow acc + mem[A]$。1960 年以前常见，如 68HC11。
Stack	`add`	0 地址，$tos \leftarrow tos + next$。1960–1970 年代常见。
Memory-Memory	`add A, B` 或 `add A, B, C`	2/3 地址，直接以内存为源和目的。1970–1980 年代常见。
Register-Memory	`add R1, A`; `load R1, A`	一个寄存器操作数和一个内存操作数，代表 80x86。
Register-Register (Load/Store)	`add R1, R2, R3`; `load R1, R2`; `store R1, R2`	ALU 只在寄存器间操作；load/store 访问内存，代表 MIPS。

操作数位置图的核心信息是：从 stack、accumulator 到 register-memory，再到 register-register/load-store，通用寄存器 (GPR) 的使用越来越突出。GPR 让操作数更多停留在处理器内部，减少内存访问。

7.2 代码序列：C = A + B 在四类指令集中的表示

Stack	Accumulator	Register-Memory	Load-Store
`Push A`	`Load A`	`Load R1, A`	`Load R1, A`
`Push B`	`Add B`	`Add R1, B`	`Load R2, B`
`Add`	`Store C`	`Store C, R1`	`Add R3, R1, R2`
`Pop C`			`Store C, R3`

这张表说明：栈和累加器指令短，但隐含状态强；register-memory 指令数较少但内存参与 ALU；load-store 指令数可能更多，但结构规则、流水线友好。

7.3 为什么几乎所有新架构都使用 GPR

通用寄存器几乎成为新 ISA 的标配，原因是：

- 寄存器比内存甚至缓存更快，寄存器值可立即获得。
- 若内存尚未准备好，处理器会等待，形成 stall。
- 寄存器方便保存变量，编译器可以把常用变量分配到寄存器中。
- 指令中寄存器编号字段很短，代码更紧凑；内存地址字段通常更长。

从处理器到存储层次看，速度与容量大致呈反向关系：寄存器最快、容量最小；缓存次之；主存更慢；磁盘最慢但容量最大。

7.4 栈架构 (Stack Architectures)

栈架构使用 First-In Last-Out (FILO) 或 Last-In First-Out (LIFO) 数据结构。ALU 操作数隐含在栈顶，算术逻辑操作通常无需显式操作数；`push`/`pop` 需要一个显式操作数。

- **优点**：指令短，代码密度好；硬件要求低；简单编译器容易编写。
- **缺点**：栈顶成为瓶颈；并行性和流水线能力弱；需要的数据不一定在栈顶，常需 `TOP`、`SWAP` 等额外指令；优化编译器难写。
- **补救思路**：为栈中值提供随机访问，或用寄存器文件/缓存保存栈顶若干项，缓解栈大小和访问延迟问题。
- **例子**：1960 年代 Burroughs B5500/B6500、HP 3000/70；现代 Java VM 采用栈式字节码模型。

7.5 累加器架构 (Accumulator Architectures)

累加器架构只有一个核心工作寄存器 ACC。指令通常包括：

- **ALU**：$Acc \leftarrow Acc + *M$。
- **Load to accumulator**：$Acc \leftarrow *M$。
- **Store from accumulator**：$*M \leftarrow Acc$。

其特点是一个显式操作数通常为内存地址，另一个隐式操作数为累加器。

- **优点**：硬件需求很低，设计简单，容易理解；指令较短，内部状态最少。
- **缺点**：累加器成为瓶颈；并行和流水线能力弱；内存流量高，需要大量 load/store。
- **例子**：早期 IBM 7090、DEC PDP-8；现代部分 DSP 架构仍有累加器思想。

7.6 内存-内存架构 (Memory-Memory Architectures)

内存-内存架构中，所有 ALU 操作数都可以直接来自内存地址。例如：

```
mul D <- C*D,    add D <- D+B
```

可直接完成 $D = B + (C \times D)$ 一类操作。

- **优点**：不浪费寄存器；若支持三操作数，指令数最低；编译器相对容易生成代码。
- **缺点**：内存流量极高；指令长度变化大；每条指令时钟数变化大；两操作数形式会产生更多数据移动。
- **例子**：VAX。

7.7 寄存器-内存架构 (Register-Memory Architectures)

寄存器-内存架构的 ALU 操作通常包含一个内存地址和一个寄存器，常见为二操作数形式。例如：

$$R1 \leftarrow R1 \times M[D], \qquad R1 \leftarrow R1 + M[B]$$

- **优点**：某些数据无需先 load 就可参与运算；指令格式相对容易编码；代码密度好，指令数较少。
- **缺点**：操作数不对等，正交性差；结果可能破坏一个源操作数；指令长度和每条指令周期数变化；更难流水线化；可能限制寄存器数量。
- **例子**：IBM 360/370、VAX、80x86。

7.8 装载-存储架构 (Load-Store Architectures)

Load-store 架构中，ALU 操作没有内存地址，通常是三操作数寄存器指令；只有 load 和 store 指令访问内存：

```
load R1 <- C,  load R2 <- D,  mul R4 <- R1*R2,  add R5 <- R4+R3
```

- **优点**：简单固定长指令编码；各指令周期数相近；容易流水线化和超标量执行；三操作数格式简化寄存器分配。
- **缺点**：指令条数较多；并非所有指令都需要三个操作数；依赖优秀编译器。
- **例子**：CDC6600、CRAY-1、大多数 RISC。

7.9 寄存器的优缺点 (Registers: Advantages and Disadvantages)

寄存器的优势与代价

优势：

- 比缓存和主存更快，不需要复杂寻址方式或标签匹配。
- 访问确定，没有 cache miss。
- 可复制多读端口，支持多个操作数同时读取。
- 标识符短，通常只需 3 到 8 位。
- 减少内存流量。

代价：

- 过程调用和上下文切换时需要保存与恢复。
- 不能像内存变量那样直接“取寄存器地址”，因此不适合作为指针目标。
- 大小固定，不适合高效存储字符串或结构体。
- 必须由编译器管理。
- 数量有限。

课件最后强调：1980 年以后设计的每一种 ISA 都采用 load-store ISA，也就是 RISC 风格，以简化 CPU 设计。

8 复习抓手与补充知识 (Review Notes)

8.1 ISA 与微体系结构的区别

- **ISA** 是软件可见接口，回答“程序员能使用哪些指令、寄存器和内存模型”。
- **Microarchitecture** 是 ISA 的实现，回答“硬件内部如何取指、译码、执行、访存、提交”。
- 同一 ISA 可有多个实现；同一程序的机器码可在这些实现上运行，但性能、功耗和内部组织不同。

8.2 评估 ISA 的常用指标

复习时可用以下指标判断一个设计选择的影响：

- **Code Size**：程序占用的指令字节数，受指令长度、操作数个数、寻址方式影响。

- **Instruction Count**：完成同一任务需要多少条指令。
- **Cycles per Instruction (CPI)**：每条指令平均执行周期。
- **Clock Cycle Time**：硬件复杂度会影响时钟周期长度，例如复杂译码可能拖慢时钟。
- **Memory Traffic**：访问内存次数越多，越容易受缓存未命中和主存延迟影响。
- **Compiler Complexity**：简单硬件常把优化压力转移给编译器。

8.3 常见易混点

- **立即数不是地址**：立即寻址使用的是指令中写死的值；直接寻址使用的是该字段指向的内存内容。
- **间接寻址多取一层**：直接寻址是 $M[K]$，间接寻址是 $M[M[K]]$。
- **RISC 不等于指令永远少**：现代 ISA 会为了新应用加入复杂指令；CISC/RISC 是设计倾向，不是绝对二分。
- **Load-store 的重点不是 "有 load/store 指令"**：关键是 ALU 运算只在寄存器间发生，访存只由 load/store 完成。
- **寄存器更快但不是免费资源**：数量有限，过程调用和上下文切换有保存恢复成本。

9 全章总结 (Summary)

本讲围绕“指令集体系结构如何定义软硬件接口”展开。复习时抓住以下主线：

1. **ISA 是契约**：它定义机器语言、寄存器、内存组织和异常模型，是高级语言经编译器、汇编器到硬件执行之间的关键接口。
2. **指令导致状态变化**：一条指令根据 opcode、操作数和寻址方式，把 PC、内存、寄存器等机器状态从 before state 变成 after state。
3. **设计必须折中**：指令短、规则、易译码有利于硬件；复杂指令和丰富寻址方式可能降低代码大小和编译难度，但增加硬件复杂性。
4. **操作数个数影响表达方式**：三地址灵活但字段多；二地址短但破坏源操作数；一地址依赖累加器；零地址依赖栈。
5. **寻址方式决定操作数定位**：立即、直接、间接、寄存器、相对、变址、基址等方式本质上都是计算有效地址的规则。
6. **CISC/RISC 是连续谱**：CISC 强调复杂指令和代码密度，RISC 强调简单规则、load-store、固定格式和流水线友好；现代处理器常混合两者思想。

7. **GPR 与 load-store 成为主流**：寄存器快、编号短、减少内存流量；1980 年后新 ISA 普遍采用 load-store 风格以简化 CPU 设计。

> “*ISA* 不是某个处理器内部怎么做，而是软件能够依赖硬件一定会做什么。”